

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

**Лабораторная работа №2
“Реализация микросервисов”**

Выполнил:

Данилова Анастасия

Группа

К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Разделить монолитное приложение rental-service на независимые микросервисы с отдельными базами данных, единой аутентификацией JWT и маршрутизацией через API Gateway, сохранив соответствие контрактам OpenAPI.

Ход работы

Архитектура решения

Состав системы

Микросервис	Порт	БД	Ответственность
auth-service	8081	auth_db	Регистрация, login, JWT, refresh, профиль
property-service	8082	property_db	Недвижимость, amenities, изображения
rental-service	8083	rental_db	Аренда, статусы, dashboard
chat-service	8084	chat_db	Чаты и сообщения
nginx (gateway)	8080	—	/api/* → сервисы

Структура

labs/rental-platform/

├── shared/ # JWT, middleware, RabbitMQ helpers

├── services/ # 4 микросервиса (Clean Architecture)

├── infra/nginx/

└── deploy/docker-compose.yml

Database per service

Идентификаторы `owner_id`, `tenant_id`, `landlord_id` хранятся как `uint` без FK на таблицы других БД. Согласованность - через HTTP internal API и события (Д35).

Синхронное взаимодействие

Откуда	Откуда	Endpoint
rental-service	property-service	GET /internal/properties/{id}
rental-service	auth-service	POST /internal/users/validate
chat-service	property-service	GET /internal/properties/{id}
property-service	auth-service	GET /internal/users/{id}

Ход выполнения

Анализ монолита и OpenAPI

Монолит (rental-service) содержал единую БД и смешанные handlers. По homeworks/hw4/*.yaml выделены границы доменов и публичные/internal контракты.

Auth			^
POST	/auth/register	Регистрация пользователя	▼
POST	/auth/login	Авторизация пользователя	▼
POST	/auth/refresh	Обновить access token	▼
POST	/auth/logout	Выход из системы	🔒 ▼
Users			^
GET	/users/me	Получить текущего пользователя	🔒 ▼
PATCH	/users/me	Обновить профиль пользователя	🔒 ▼
DELETE	/users/me	Удалить аккаунт (soft delete)	🔒 ▼
PUT	/users/change-password	Смена пароля	🔒 ▼
GET	/users	Получить список пользователей	🔒 ▼
GET	/users/{user_id}	Получить пользователя по ID	🔒 ▼
Internal			^
GET	/internal/users/{user_id}	Получить пользователя для межсервисного взаимодействия	▼
POST	/internal/users/validate	Проверка существования пользователей	▼

auth_service API

Chats			^
GET	/chats	Получить список чатов пользователя	🔒 ▼
POST	/chats	Создать чат по объекту недвижимости	🔒 ▼
GET	/chats/{chat_id}	Получить чат	🔒 ▼
DELETE	/chats/{chat_id}	Удалить чат	🔒 ▼
Messages			^
GET	/chats/{chat_id}/messages	Получить сообщения чата	🔒 ▼
POST	/chats/{chat_id}/messages	Отправить сообщение	🔒 ▼
PATCH	/messages/{message_id}	Изменить сообщение	🔒 ▼
DELETE	/messages/{message_id}	Удалить сообщение	🔒 ▼
PATCH	/chats/{chat_id}/read	Пометить сообщения как прочитанные	🔒 ▼
Internal			^
GET	/internal/chats/property/{property_id}	Получить количество чатов объекта	▼

chat_service API

Properties			^
GET	/properties	Получить список объектов недвижимости	▼
POST	/properties	Создать объект недвижимости	🔒 ▼
GET	/properties/{property_id}	Получить объект недвижимости	▼
PATCH	/properties/{property_id}	Обновить объект недвижимости	🔒 ▼
DELETE	/properties/{property_id}	Удалить объект недвижимости	🔒 ▼
GET	/properties/me	Получить мои объявления	🔒 ▼
PropertyImages			^
GET	/properties/{property_id}/images	Получить изображения объекта	▼
POST	/properties/{property_id}/images	Добавить изображение	🔒 ▼
DELETE	/images/{image_id}	Удалить изображение	🔒 ▼
Amenities			^
GET	/amenities	Получить список amenities	▼
POST	/amenities	Создать amenity	🔒 ▼
PATCH	/amenities/{amenity_id}	Обновить amenity	🔒 ▼
DELETE	/amenities/{amenity_id}	Удалить amenity	🔒 ▼
Internal			^
GET	/internal/properties/{property_id}	Получить property для межсервисного взаимодействия	▼

property_service API

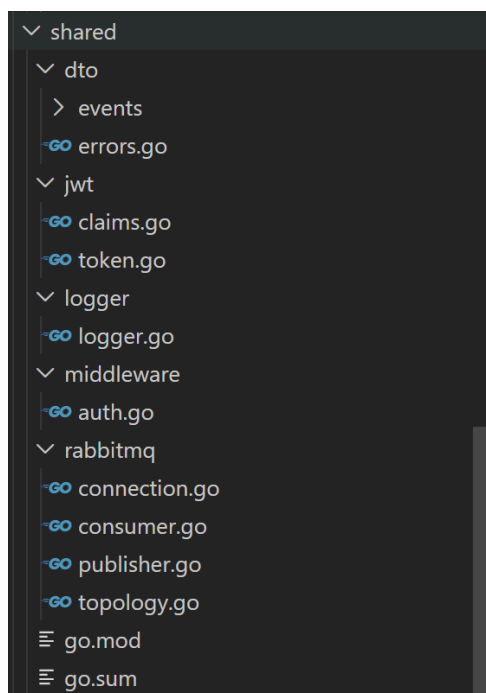
Rentals			^
GET	/rentals	Получить список моих аренд	🔒 ▼
POST	/rentals	Создать заявку на аренду	🔒 ▼
GET	/rentals/{rental_id}	Получить договор аренды	🔒 ▼
DELETE	/rentals/{rental_id}	Отменить аренду	🔒 ▼
PATCH	/rentals/{rental_id}/approve	Подтвердить аренду	🔒 ▼
PATCH	/rentals/{rental_id}/reject	Отклонить заявку	🔒 ▼
PATCH	/rentals/{rental_id}/complete	Завершить аренду	🔒 ▼
Dashboard			^
GET	/dashboard	Личный кабинет пользователя	🔒 ▼
Internal			^
GET	/internal/rentals/property/{property_id}/active	Проверить активную аренду объекта	▼

property_service API

Общий модуль shared

- shared/jwt - claims, подпись и валидация access token
- shared/middleware - JWTAuth для всех сервисов
- shared/rabbitmq, shared/dto/events - задел под асинхронное взаимодействие

(Д35)



структура shared

Auth Service

Реализованы эндпоинты: register, login, refresh, logout, /users/me, change-password, soft delete, internal validate/get user.

POST 0. E2E Main Flow / 0.1 Register Landlord http://localhost:8080/api/auth/register	201 - 182 ms - 414 B - 1
PASS 201 Created	
POST 0. E2E Main Flow / 0.2 Register Tenant http://localhost:8080/api/auth/register	201 - 88 ms - 489 B - 1
PASS 201 Created	
POST 0. E2E Main Flow / 0.3 Login Landlord http://localhost:8080/api/auth/login	200 - 88 ms - 431 B - 1
PASS 200 OK	
POST 0. E2E Main Flow / 0.4 Login Tenant http://localhost:8080/api/auth/login	200 - 186 ms - 438 B - 1
PASS 200 OK	
POST 0. E2E Main Flow / 0.5 Landlord — Create Property http://localhost:8080/api/properties	201 - 34 ms - 433 B - 1
PASS 201 Created	
GET 0. E2E Main Flow / 0.6 Tenant — List Properties http://localhost:8080/api/properties?city=Saint&is_available=true	200 - 23 ms - 516 B - 2
PASS 200 OK	
PASS array	
POST 0. E2E Main Flow / 0.7 Tenant — Create Rental http://localhost:8080/api/rentals	201 - 58 ms - 636 B - 2
PASS 201 Created	
PASS status PENDING	
PATCH 0. E2E Main Flow / 0.8 Landlord — Approve Rental http://localhost:8080/api/rentals/2/approve	200 - 19 ms - 627 B - 2
PASS 200 OK	
PASS status ACTIVE	
POST 0. E2E Main Flow / 0.9 Tenant — Create Chat http://localhost:8080/api/chats	201 - 57 ms - 296 B - 1
PASS 201 or 200	
POST 0. E2E Main Flow / 0.10 Tenant — Send Message http://localhost:8080/api/chats/2/messages	201 - 13 ms - 371 B - 1
PASS 201 Created	
GET 0. E2E Main Flow / 0.11 Tenant — Dashboard http://localhost:8080/api/dashboard	200 - 8 ms - 398 B - 2
PASS 200 OK	
PASS {user: {id: 1, email: 'landlord@e2e.com', password: '123456', role: 'LANDLORD', is_active: true, is_verified: true, created_at: '2023-10-26T10:00:00.000Z', updated_at: '2023-10-26T10:00:00.000Z'}, properties: [{id: 1, title: 'Property 1', description: 'Property 1 description', city: 'Saint', is_available: true, created_at: '2023-10-26T10:00:00.000Z', updated_at: '2023-10-26T10:00:00.000Z'}]}	

Property Service

CRUD properties, amenities, images; поле is_available; internal property для rental/chat.

GET 3. Property Service / Get Property by ID http://localhost:8080/api/properties/3 No tests found	200 • 7 ms • 428 B
GET 3. Property Service / My Properties (Landlord) http://localhost:8080/api/properties/me No tests found	200 • 6 ms • 286 B
GET 3. Property Service / Filter by type and price http://localhost:8080/api/properties?property_type=APARTMENT&min_price=10000&max_price=100000 No tests found	200 • 7 ms • 516 B
POST 3. Property Service / Create Amenity http://localhost:8080/api/amenities No tests found	409 • 9 ms • 203 B
GET 3. Property Service / List Amenities http://localhost:8080/api/amenities No tests found	200 • 7 ms • 193 B
POST 3. Property Service / Add Property Image http://localhost:8080/api/properties/3/images No tests found	201 • 14 ms • 243 B

Rental Service

Создание заявки, approve/reject/complete, GET /dashboard (агрегация без отдельного BFF).

GET 4. Rental Service / Get Rental http://localhost:8080/api/rentals/2 No tests found	200
GET 4. Rental Service / List Rentals (tenant) http://localhost:8080/api/rentals?role=tenant No tests found	200
PATCH 4. Rental Service / Complete Rental http://localhost:8080/api/rentals/2/complete No tests found	200

Chat Service

GET 5. Chat Service / List Chats

http://localhost:8080/api/chats

200

No tests found

GET 5. Chat Service / List Messages

http://localhost:8080/api/chats/2/messages?limit=50&offset=0

200

No tests found

PATCH 5. Chat Service / Mark Chat Read

http://localhost:8080/api/chats/2/read

200

No tests found

API Gateway

nginx маршрутизирует /api/auth/*, /api/properties/*, /api/rentals/*, /api/chats/*, /api/dashboard.

```
1 upstream auth_upstream {
2     server auth-service:8081;
3 }
4
5 upstream property_upstream {
6     server property-service:8082;
7 }
8
9 upstream rental_upstream {
10    server rental-service:8083;
11 }
12
13 upstream chat_upstream {
14    server chat-service:8084;
15 }
16
17 server {
18     listen 80;
19
20     location = /health {
21         return 200 '{"status":"ok","gateway":true}';
22         add_header Content-Type application/json;
23     }
24
25     # Auth: /api/auth/login -> /auth/login
26     location /api/auth/ {
27         rewrite ^/api/auth/(.*)$ /auth/$1 break;
28         proxy_pass http://auth_upstream;
29         proxy_set_header Host $host;
30         proxy_set_header X-Real-IP $remote_addr;
31         proxy_set_header Authorization $http_authorization;
32     }
33
34     # Users (auth service): /api/users/me -> /users/me
35     location /api/users/ {
36         rewrite ^/api/users/(.*)$ /users/$1 break;
37         proxy_pass http://auth_upstream;
38         proxy_set_header Host $host;
39         proxy_set_header X-Real-IP $remote_addr;
40         proxy_set_header Authorization $http_authorization;
41     }
42
43     # Properties
44     location /api/properties/ {
45         rewrite ^/api/properties/(.*)$ /properties/$1 break;
46         proxy_pass http://property_upstream;
47         proxy_set_header Host $host;
48         proxy_set_header Authorization $http_authorization;
49     }
50
51     location = /api/properties {
52         rewrite ^/api/properties$ /properties break;
53         proxy_pass http://property_upstream;
54         proxy_set_header Host $host;
55         proxy_set_header Authorization $http_authorization;
```

Тестирование

Коллекция Postman: labs/lab2/postman/Rental Platform
API.postman_collection.json

Подробное описание: labs/lab2/TESTING.md

Сравнение с монолитом

Аспект	Монолит (lab1)	Микросервисы (lab2)
БД	Одна PostgreSQL	4 отдельные БД
Деплой	Один процесс	gateway
Границы	Пакеты в одном модуле	Отдельные go.mod
Контракт	hw2 endpoints.yaml	hw4 per-service OpenAPI

Монолит сохранён в labs/lab1/rental-service/ без изменений.

Выводы

1. Бэкенд разделён на 4 микросервиса с изолированными БД и контрактами OpenAPI hw4.
2. JWT централизован в auth-service; остальные сервисы валидируют токен через shared/middleware.
3. Синхронные проверки выполняются через internal REST без межсервисных FK.
4. Клиентский доступ унифицирован через nginx на порту 8080.
5. Тестирование автоматизировано Postman-коллекцией в labs/lab2/postman/.